Matthew Tran
Secret-Key Encryption: Modes, Padding, and Vulnerabilities

In this lab I practiced secret key encryption using OpenSSL and Python. I encrypted text and images with different modes like ECB and CBC. I saw how ECB leaves patterns visible while CBC hides them better. I also learned why padding is needed and why reusing an Initialization Vector is a security risk.

Tools Used

1. OpenSSL
2. Linux command-lines
3. Python
4. Ubuntu Linux Virtual Machine

Skills Learned

1. Encrypting and decrypting files using OpenSSL
2. Understanding block ciphers vs. stream cipher modes
3. Visualizing data leakage in ECB mode
4. Identifying when and why padding is required
5. Inspecting PKCS padding at the byte level
6. Understanding the role of IVs in CBC mode
7. Exploiting IV reuse using XOR operations

Task 1) Encrypting Data Using Different Ciphers and Modes

1. Opened the terminal inside my manually created virtual machine.
2. Navigated to the directory containing article.txt using the cd command.
3. Encrypted article.txt using AES-128-CBC with a manually specified key and IV using the command: openssl enc -aes-128-cbc -e -in article.txt -out article_aes_cbc.bin -K <key> -iv <iv>.
4. Encrypted the same file using AES-128-CFB using: openssl enc -aes-128-cfb -e -in article.txt -out article_aes_cfb.bin -K <key> -iv <iv>.
5. Encrypted the file again using Blowfish in CBC mode with openssl enc -bf-cbc -e -in article.txt -out article_bf_cbc.bin -K <key> -iv <iv>.

6. Listed the encrypted output files using ls -l *.bin to confirm they were created.

```
matt@matt-VirtualBox:~/secret_key_lab$ K=00112233445566778899aabbccddeeff
matt@matt-VirtualBox:~/secret_key_lab$ IV=0102030405060708090a0b0c0d0e0f10
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-cbc -in article.txt
 -out article_aes128cbc.bin -K $K -iv $IV
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -bf-cbc -in article.txt -out
 article_bfcbc.bin -K 0123456789abcdef0123456789abcdef -iv 0102030405060708
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-cfb -in article.txt
 -out article_aes128cfb.bin -K $K -iv $IV
matt@matt-VirtualBox:~/secret_key_lab$ ls -l article_*.bin
-rw-rw-r-- 1 matt matt 1872 Oct 17 17:16 article_aes128cbc.bin
-rw-rw-r-- 1 matt matt 1865 Oct 17 17:16 article_aes128cfb.bin
-rw-rw-r-- 1 matt matt 1872 Oct 17 17:16 article_bfcbc.bin
```

This task showed that encrypting the same plaintext using different encryption algorithms and modes produces completely different ciphertexts. It helped me understand how cipher choice and encryption mode directly impact how data is protected.

Task 2) Encrypting an Image Using ECB and CBC Modes

1. Verified that pic_original.bmp existed in the working directory using ls.
2. Encrypted the image using AES-128 in ECB mode with: openssl enc -aes-128-ecb -e -in pic_original.bmp -out encrypted_ecb.bmp -K <key>.
3. Encrypted the same image using AES-128 in CBC mode with: openssl enc -aes-128-cbc -e -in pic_original.bmp -out encrypted_cbc.bmp -K <key> -iv <iv>.

```
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-ecb -in pic_origina
l.bmp -out encrypted_ecb.bmp -K $K
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-cbc -in pic_origina
l.bmp -out encrypted_cbc.bmp -K $K -iv $IV
matt@matt-VirtualBox:~/secret_key_lab$ ls -l encrypted_ecb.bmp encrypted_cbc.bm
p
-rw-rw-r-- 1 matt matt 184976 Oct 17 17:19 encrypted_cbc.bmp
-rw-rw-r-- 1 matt matt 184976 Oct 17 17:19 encrypted_ecb.bmp
```

This task prepared the image files for comparison and demonstrated how ECB mode does not use an IV, while CBC mode relies on one to introduce randomness.

Task 3) Rebuilding Encrypted Images into Valid Bitmap Files

1. Extracted the first 54 bytes (bitmap header) from the original image using: head -c 54 pic_original.bmp > header.bmp.

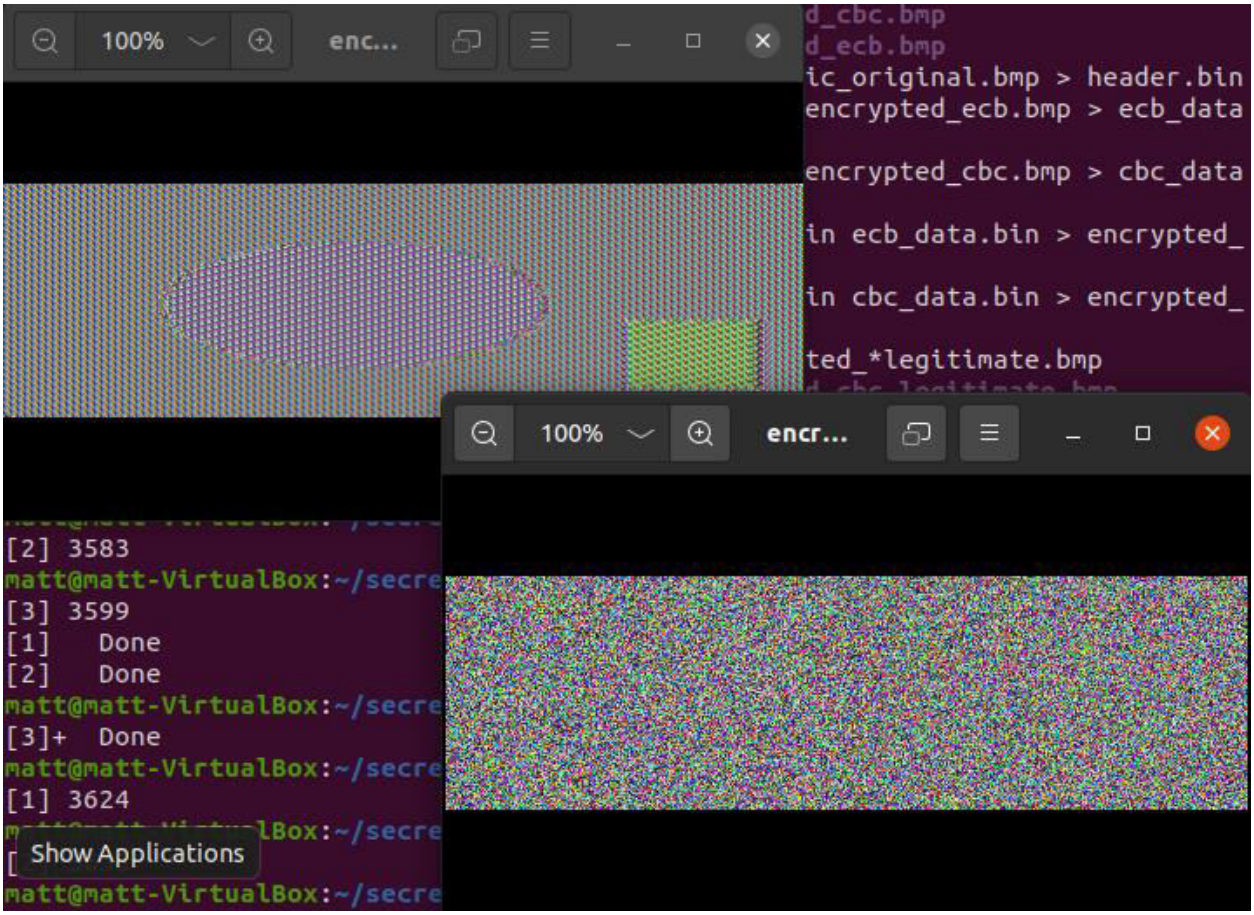
2. Extracted the encrypted image data from the ECB-encrypted file using: tail -c +55 encrypted_ecb.bmp > ecb_data.bin.
3. Combined the original header and encrypted ECB data using: cat header.bmp ecb_data.bin > encrypted_ecb_legitimate.bmp.
4. Repeated the same process for the CBC-encrypted file to create encrypted_cbc_legitimate.bmp.

```
matt@matt-VirtualBox:~/secret_key_lab$ head -c 54 pic_original.bmp > header.bin
matt@matt-VirtualBox:~/secret_key_lab$ tail -c +55 encrypted_ecb.bmp > ecb_data
.bin
matt@matt-VirtualBox:~/secret_key_lab$ tail -c +55 encrypted_cbc.bmp > cbc_data
.bin
matt@matt-VirtualBox:~/secret_key_lab$ cat header.bin ecb_data.bin > encrypted_
ecb_legitimate.bmp
matt@matt-VirtualBox:~/secret_key_lab$ cat header.bin cbc_data.bin > encrypted_
cbc_legitimate.bmp
matt@matt-VirtualBox:~/secret_key_lab$ ls -l encrypted_*legitimate.bmp
-rw-rw-r-- 1 matt matt 184976 Oct 17 17:19 encrypted_cbc_legitimate.bmp
-rw-rw-r-- 1 matt matt 184976 Oct 17 17:19 encrypted_ecb_legitimate.bmp
```

This step was necessary because bitmap files require a valid header to be displayed. By reusing the original header, I was able to view the encrypted image data correctly.

Task 4) Visual Comparison of ECB and CBC Encrypted Images

1. Opened the reconstructed ECB image using: eog encrypted_ecb_legitimate.bmp.
2. Opened the reconstructed CBC image using: eog encrypted_cbc_legitimate.bmp.



The ECB-encrypted image still revealed the structure and outline of the original image, while the CBC-encrypted image appeared as random noise. This showed that ECB mode leaks visual patterns and is insecure for structured data.

Task 5) Identifying Encryption Modes That Require Padding

1. Used the commands show in the image below.
2. Compared output file sizes using ls -l article_*.bin.

```
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-ecb -in article.txt
  -out art_ecb.bin  -K $K
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-cbc -in article.txt
  -out art_cbc.bin  -K $K -iv $IV
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-cfb -in article.txt
  -out art_cfb.bin  -K $K -iv $IV
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-ofb -in article.txt
  -out art_ofb.bin  -K $K -iv $IV
matt@matt-VirtualBox:~/secret_key_lab$ ls -l art_*.bin
-rw-rw-r-- 1 matt matt 1872 Oct 17 18:22 art_cbc.bin
-rw-rw-r-- 1 matt matt 1865 Oct 17 18:22 art_cfb.bin
-rw-rw-r-- 1 matt matt 1872 Oct 17 18:22 art_ecb.bin
-rw-rw-r-- 1 matt matt 1865 Oct 17 18:22 art_ofb.bin
```

The ecb and cbc required padding because of the slightly larger output. They both had an output of 1872 whereas cfb and ofb had a size of 1865. The ecb and cbc files padded to make the plaintext a multiple of the AES block size (they are block ciphers). The cfb and ofb files are smaller because they are stream modes, that only need to encrypt exactly as many bytes as the input.

Task 6) Creating Files of Specific Sizes

1. Created a 5-byte file using: echo -n "abcde" > f1.txt.
2. Created a 10-byte file using: echo -n "abcdefghij" > f2.txt.
3. Created a 16-byte file using: echo -n "abcdefghijklmnop" > f3.txt.
4. Verified the file sizes using ls -l.

```
matt@matt-VirtualBox:~/secret_key_lab$ echo -n "12345" > f1.txt
matt@matt-VirtualBox:~/secret_key_lab$ echo -n "1234567890" > f2.txt
matt@matt-VirtualBox:~/secret_key_lab$ echo -n "1234567890ABCDEF" > f3.txt
matt@matt-VirtualBox:~/secret_key_lab$ ls -l f1.txt f2.txt f3.txt
-rw-rw-r-- 1 matt matt  5 Oct 17 18:46 f1.txt
-rw-rw-r-- 1 matt matt 10 Oct 17 18:47 f2.txt
-rw-rw-r-- 1 matt matt 16 Oct 17 18:47 f3.txt
```

These files were used to clearly observe how padding behaves when encrypting data of different lengths relative to the AES block size.

Task 7) Inspecting PKCS Padding Behavior

1. Encrypted all three files using AES-128-CBC.
2. Decrypted each file using the -nopad option to preserve padding bytes.
3. Used the hexdump -C command to view the decrypted output in hexadecimal format.

```
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-cbc -in f1.txt -out
 f1_enc.bin -K $K -iv $IV -nosalt
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-cbc -in f2.txt -out
 f2_enc.bin -K $K -iv $IV -nosalt
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-cbc -in f3.txt -out
 f3_enc.bin -K $K -iv $IV -nosalt
matt@matt-VirtualBox:~/secret_key_lab$ ls -l f*_enc.bin
-rw-rw-r-- 1 matt matt 16 Oct 17 18:48 f1_enc.bin
-rw-rw-r-- 1 matt matt 16 Oct 17 18:48 f2_enc.bin
-rw-rw-r-- 1 matt matt 32 Oct 17 18:48 f3_enc.bin
```

```
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-cbc -d -in f1_enc.b
in -out f1_decrypted_nopad.bin -K $K -iv $IV -nopad -nosalt
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-cbc -d -in f2_enc.b
in -out f2_decrypted_nopad.bin -K $K -iv $IV -nopad -nosalt
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-cbc -d -in f3_enc.b
in -out f3_decrypted_nopad.bin -K $K -iv $IV -nopad -nosalt
matt@matt-VirtualBox:~/secret_key_lab$ hexdump -C f1_decrypted_nopad.bin
00000000  31 32 33 34 35 0b 0b 0b  0b 0b 0b 0b 0b 0b 0b 0b  |12345...........|
00000010
matt@matt-VirtualBox:~/secret_key_lab$ hexdump -C f2_decrypted_nopad.bin
00000000  31 32 33 34 35 36 37 38  39 30 06 06 06 06 06 06  |1234567890......|
00000010
matt@matt-VirtualBox:~/secret_key_lab$ hexdump -C f3_decrypted_nopad.bin
00000000  31 32 33 34 35 36 37 38  39 30 41 42 43 44 45 46  |1234567890ABCDEF|
00000010  10 10 10 10 10 10 10 10  10 10 10 10 10 10 10 10  |................|
00000020
```

The results showed that files smaller than 16 bytes were padded to a full block, and a file that was exactly 16 bytes long received an additional full block of padding. This confirmed how PKCS padding works.

Task 8) Demonstrating the Importance of IV Uniqueness

1. Encrypted article.txt using CBC mode with one IV.
2. Encrypted the same file again using a different IV.
3. Re-encrypted the file using the original IV.
4. Compared the resulting ciphertext files.

```
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-cbc -e -in article.txt -out enc1_diffIV.bi
n \
> -K 00112233445566778899AABBCCDDEEFF -iv 00000000000000000000000000000000 -nosalt
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-cbc -e -in article.txt -out enc2_diffIV.bi
n \
> -K 00112233445566778899AABBCCDDEEFF -iv FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF -nosalt
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-cbc -e -in article.txt -out enc1_sameIV.bi
n \
> -K 00112233445566778899AABBCCDDEEFF -iv 1234567890ABCDEF1234567890ABCDEF -nosalt
matt@matt-VirtualBox:~/secret_key_lab$ openssl enc -aes-128-cbc -e -in article.txt -out enc2_sameIV.bi
n \
> -K 00112233445566778899AABBCCDDEEFF -iv 1234567890ABCDEF1234567890ABCDEF -nosalt
matt@matt-VirtualBox:~/secret_key_lab$ hexdump -C enc1_diffIV.bin | head -n 2
00000000  9b ce b6 2d 94 d2 53 83  74 c5 1e d9 b0 34 27 19  |...-..S.t....4'.|
00000010  68 31 69 cf 32 e3 d1 ec  da 32 ba f2 bd 10 54 2e  |h1i.2....2....T.|
matt@matt-VirtualBox:~/secret_key_lab$ hexdump -C enc2_diffIV.bin | head -n 2
00000000  48 18 ac e5 d7 15 65 7d  ef c6 2f 95 68 13 e2 ce  |H.....e}../.h...|
00000010  b2 21 a3 1c 0d a3 75 29  b2 0a 9d f3 25 2f 05 72  |.!....u)....%/.r|
matt@matt-VirtualBox:~/secret_key_lab$ hexdump -C enc1_sameIV.bin | head -n 2
00000000  aa a1 cd f2 5f 53 e4 42  50 9f 9b 19 8c cf 79 a1  |...._S.BP.....y.|
00000010  ad cd ed 46 d5 ea 22 3e  e0 f0 d0 51 c2 85 4b ff  |...F..">...Q..K.|
matt@matt-VirtualBox:~/secret_key_lab$ hexdump -C enc2_sameIV.bin | head -n 2
00000000  aa a1 cd f2 5f 53 e4 42  50 9f 9b 19 8c cf 79 a1  |...._S.BP.....y.|
00000010  ad cd ed 46 d5 ea 22 3e  e0 f0 d0 51 c2 85 4b ff  |...F..">...Q..K.|
```

When the same IV was reused, the ciphertexts were identical. This demonstrated that IV reuse weakens security and must be avoided.

Task 9) Exploiting IV Reuse Using XOR

1. Created and edited a Python script named iv_attack.py.
2. Used XOR operations to compute T = P1 ⊕ C1 and recover the unknown plaintext using P2 = T ⊕ C2.
3. Ran the script using python3 iv_attack.py.

```
matt@matt-VirtualBox:~/secret_key_lab$ vim iv_attack.py
matt@matt-VirtualBox:~/secret_key_lab$ python3 iv_attack.py
Recovered P2: b'Order: Launch a missile!'
Decoded: Order: Launch a missile!
```

```
P1 = b"This is a known message!"
C1_hex = "a469b1c502c1cab966965e50425438e1bb1b5f9037a4c159"
C2_hex = "bf73bcd3509299d566c35b5d450337e1bb175f903fafc159"

C1 = bytes.fromhex(C1_hex)
C2 = bytes.fromhex(C2_hex)

T = bytes(p ^ c for p, c in zip(P1, C1))
P2_recovered = bytes(t ^ c for t, c in zip(T, C2))

print("Recovered P2:", P2_recovered)
print("Decoded:", P2_recovered.decode('utf-8'))
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
"iv_attack.py" 12L, 387C                                    1
```

This task showed that reusing an IV allows an attacker to recover plaintext without knowing the encryption key, proving how dangerous improper IV usage can be.